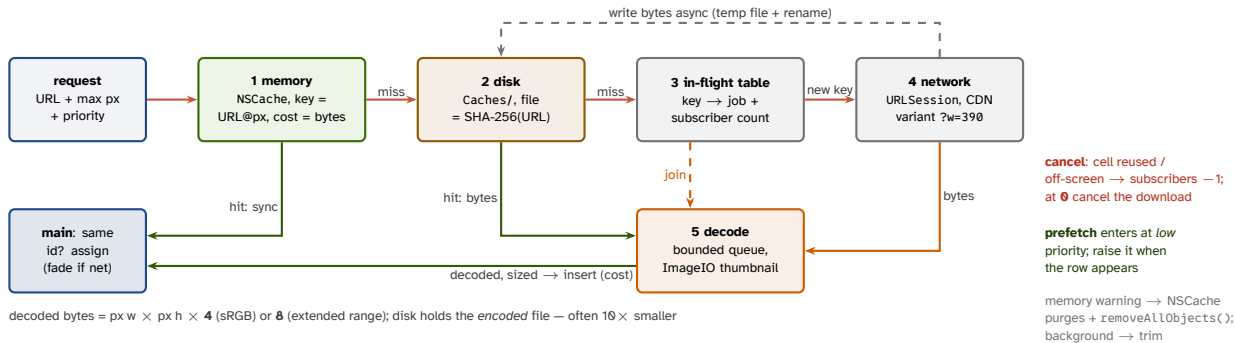


In one line: A request walks **memory** (decoded, *sized* bitmaps) → **disk** (encoded bytes) → **network** (deduplicated, prioritised, cancellable) and comes back through **decode + downsample off the main thread**. What matters: the cache **key**, the **memory maths**, **one download per key** with cancellation only when the *last* subscriber leaves, an **identity check** before assigning, and **prefetch** at low priority.



How it works

- Key** = URL + target *pixel* size (+ transforms: crop, rounding, blur). Same URL at 2 sizes = 2 memory entries; disk keeps one set of bytes per URL.
- Memory**: NSCache<Key, UIImage> — thread-safe, keys not copied, evicts on its own under pressure. Set `totalCostLimit` with `cost = decoded bytes`; `countLimit` alone lets 10 huge images blow the budget. Apple: eviction order “not guaranteed”, limit “not strict” — so not an LRU; size it, don’t rely on order.
- Disk**: encoded bytes in Caches/ (purgeable, not backed up); index of size + last-access; sweep to a byte cap + max age, off main (launch, background); write atomically. **URLCache vs custom**: URLCache gives HTTP semantics (Cache-Control, ETag revalidation) but only stores 2xx responses whose headers allow it and that are ≤ ~5% of its disk capacity, and you do not choose eviction. Custom: your key, LRU, size cap, pinning of prefetched items. Neither holds a *decoded* tier.
- Network**: **coalesce** — N cells asking for one key share one job; each caller is a subscriber; cancel the transfer only when the count reaches 0. **Priority**: URLSessionTask.priority (0-1; lowPriority 0.25, highPriority 0.75; only a *hint*) — visible high, prefetch low. Cap concurrency. Ask the CDN for the size you need (?w=390, Accept: image/webp) — the cheapest byte is the one never sent.
- Decode**: off main on a *bounded* queue (WWDC18: a serial queue, not one global-queue block per image → thread explosion). ImageIO `CGImageSourceCreateThumbnailAtIndex + kCGImageSourceThumbnailMaxPixelSize`, or iOS 15 `byPreparingThumbnail(ofSize:)`. A plain `UIImage(data:)` decodes *later*, full size, on main, in the commit’s prepare phase.
- Cancellation**: `prepareForReuse` / `didEndDisplaying`; SwiftUI `.task(id: url)` cancels on disappear or id change. Then **check identity** before assigning — a late result must not land in a reused cell.
- Prefetch**: UICollectionViewDataSourcePrefetching (iOS 10) `prefetchItemsAt / cancelPrefetchingForItemsAt`; it is a *hint* — not called for every row on a fast fling, so display must work without it. SwiftUI lazy stacks have no prefetch hook: warm row $n+k$ from row n ’s onAppear.
- Placeholders**: dominant colour or **BlurHash** (a ~20-30 char string in the JSON, decoded to a tiny image) → progressive JPEG (`CGImageSourceCreateIncremental`) → final. Fade in only on a network load, never on a memory hit (flicker on every scroll).
- Formats**: HEIC iOS 11 · WebP decode iOS 14 · AVIF iOS 16 (*unverified*) · JPEG XL iOS 17. Check at runtime: `CGImageSourceCopyTypeIdentifiers()`. Animated GIF/WebP = one bitmap per frame in memory.

Example — the cell side

```
final class PhotoCell: UICollectionViewCell { // @MainActor
    private var load: Task<Void, Never>; private var shownID: Photo.ID?
    func show(_ p: Photo, _ pipe: ImagePipeline) {
        shownID = p.id; imageView.image = p.blurHash // instant
        let px = max(bounds.width, bounds.height)
            * traitCollection.displayScale // PIXELS
        load = Task { // inherits main actor
            let img = try? await pipe.image(p.url, maxPixel: px, .high)
            guard !Task.isCancelled, shownID == p.id, let img else { return }
            imageView.image = img // right cell, main
        }
        override func prepareForReuse() {
            super.prepareForReuse(); load?.cancel(); imageView.image = nil
        }
    }
}
```

pipe.image must translate the caller’s cancellation into “subscriber -1” — cancelling the shared task would kill the other cells’ load.

Memory maths — say the numbers

image	decoded	× 60 cells
12 MP photo, 4032 × 3024	4032·3024·4 = 48.6 MB	2.9 GB — jetsam
3-col grid, 130 pt @3x = 390 px	390·390·4 = 608 KB	36 MB
same, extended range (8 B/px)	1.2 MB	73 MB

Traps

- ! Memory cache of *encoded* data or full-size `UIImage(data:)` — decode on main, every first display, at full resolution.
- ! Cancelling the download when *one* of three cells showing it is reused.
- ! Prefetch at default priority — it competes with what is on screen.
- ! 50 decodes dispatched at once on global queues — thread explosion, worse latency.
- ! “Just use URLCache” — no decoded tier, the ~5% size rule, eviction you do not control.
- ! Fade-in animation on a memory hit; no identity check → wrong image flashes.

Remember

Key · tier · dedupe · decode small off main · check identity · prefetch low. Memory = pixels × 4; disk = bytes; network = the size you need.

Related: ios-client-system-design (feed) · rendering-pipeline (downsample code, lazy decode) · instruments-performance (memory) · collectionview-compositional · data-transfer-optimisation (formats)